

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3.7

MIGRATE FROM LEGACY AUTH

 Copy as Markdown

Migrate from legacy serverpod_auth

This guide is for apps still running `serverpod_auth_server` on Serverpod 3.4 or later. At the end, your server runs the new modular auth stack. Email users sign in with their old passwords, and existing sessions keep working. Plan for about an hour, plus migration runtime.

Two limits apply:

- **Google, Apple, and Firebase accounts** aren't linked automatically. Read the warning in [Wire up sign-in for migrated users](#) before you begin.
- **Old client builds** can still sign in by email. The end of [Configure the server](#) lists the limits on their other auth calls.

WARNING

The `serverpod_auth_bridge` and `serverpod_auth_migration` packages are experimental. They may receive breaking changes and are not yet production-ready.

Before you start

- A Serverpod 4.0.x project. If you are on an earlier version, follow [Upgrade to 4.0](#) first.
- Dart SDK 3.12.2 or later.
- Flutter SDK 3.44.4 or later (only if you are migrating the Flutter app).
- Postgres 14 or later, or SQLite3.
- The new auth packages at `4.0.0`: the server, client, and Flutter variants of `serverpod_auth_core`, `serverpod_auth_idp`, and `serverpod_auth_bridge`, plus the server and client variants of `serverpod_auth_migration`. [Add the new auth packages](#) shows which ones each pubspec needs.
- Back up your production database.



- Commit your current state on a clean branch.
- Restore a copy of production data into a staging environment and rehearse this guide against it before running it for real.

Add the new auth packages

Add the new packages to each of your server, client, and Flutter `pubspec.yaml` files. Keep the legacy `serverpod_auth_*` packages installed; you will remove them after the migration completes.

In `<project>_server/pubspec.yaml`:

```
dependencies:
  serverpod: 4.0.0
  serverpod_auth_server: 4.0.0           # legacy, keep during migration
  serverpod_auth_core_server: 4.0.0
  serverpod_auth_idp_server: 4.0.0
  serverpod_auth_bridge_server: 4.0.0
  serverpod_auth_migration_server: 4.0.0
```

In `<project>_client/pubspec.yaml`:

```
dependencies:
  serverpod_client: 4.0.0
  serverpod_auth_core_client: 4.0.0
  serverpod_auth_idp_client: 4.0.0
  serverpod_auth_bridge_client: 4.0.0
  serverpod_auth_migration_client: 4.0.0
```

In `<project>_flutter/pubspec.yaml`:

```
dependencies:
  serverpod_flutter: 4.0.0
  serverpod_auth_core_flutter: 4.0.0
  serverpod_auth_idp_flutter: 4.0.0
  serverpod_auth_bridge_flutter: 4.0.0
```

`serverpod_auth_bridge_client` and `serverpod_auth_bridge_flutter` are required for the session import covered later under [Update the Flutter app](#). The client also needs `serverpod_auth_migration_client`, because `serverpod generate` imports the client of every module the server depends on.

From each package directory, run:

```
dart pub upgrade
```

```
serverpod generate
```

The project now builds with the new packages installed alongside the legacy ones.

Configure the server

In `<project>_server/lib/server.dart`, call `pod.initializeAuthServices` before `pod.start` with the modular identity providers, register `LegacySessionTokenManager` so existing tokens keep validating, and enable legacy client forwarding so old client builds keep working.

`serverpod_auth_bridge_server` exports its own `Endpoints` and `Protocol` classes from its generated code, which clash with your project's. Use a `show` clause to bring only the symbols you need into scope:

```
import 'package:serverpod_auth_bridge_server/serverpod_auth_bridge_server.dart'
  show LegacySessionTokenManager, LegacyClientSupport;
import 'package:serverpod_auth_idp_server/core.dart';
import 'package:serverpod_auth_idp_server/providers/email.dart';
import 'package:serverpod_auth_idp_server/providers/google.dart';

import 'src/generated/serverpod.dart';

void run(List<String> args) async {
  final pod = Serverpod(args);

  pod.initializeAuthServices(
    tokenManagerBuilders: [
      JwtConfigFromPasswords(),
      ServerSideSessionsConfigFromPasswords(),
      const LegacySessionTokenManager(),
    ],
    identityProviderBuilders: [
      EmailIdpConfigFromPasswords(),
      GoogleIdpConfigFromPasswords(),
    ],
  );

  pod.enableLegacyClientSupport();

  await pod.start();
}
```

Put the password-importing email endpoint in its own file under

`<project>_server/lib/src/endpoints/`. Importing the full bridge package alongside the project's `Endpoints` and `Protocol` from the same file confuses the `serverpod generate` endpoint scanner, so keep it isolated:

```
// lib/src/endpoints/password_importing_email_idp_endpoint.dart
```

```

import 'package:serverpod/serverpod.dart';
import 'package:serverpod_auth_bridge_server/serverpod_auth_bridge_server.dart';
import 'package:serverpod_auth_idp_server/providers/email.dart';

class PasswordImportingEmailIdpEndpoint extends EmailIdpBaseEndpoint {
  @override
  Future<AuthSuccess> login(
    Session session, {
    required String email,
    required String password,
  }) async {
    try {
      return await super.login(session, email: email, password: password);
    } on EmailAccountLoginException catch (e) {
      if (e.reason != EmailAccountLoginExceptionReason.invalidCredentials) {
        rethrow;
      }
      final account = await emailIdp.admin.findAccount(session, email: email);
      if (account == null || account.hasPassword) {
        rethrow;
      }
    }

    await AuthBackwardsCompatibility.importLegacyPasswordIfNeeded(
      session,
      email: email,
      password: password,
    );
    return super.login(session, email: email, password: password);
  }
}

```

If your project already has an endpoint that extends `EmailIdpBaseEndpoint`, such as `lib/src/auth/email_idp_endpoint.dart` from the project template, delete it. With two such endpoints, the Flutter email sign-in UI throws `ServerpodClientMultipleEndpointsFound`.

Add the required entries to `<project>_server/config/passwords.yaml` under the `development:` section (use distinct values per environment):

```

development:
  # ... existing keys (database, redis, serviceSecret) ...
  jwtRefreshTokenHashPepper: 'your-jwt-refresh-pepper'
  jwtHmacSha512PrivateKey: 'your-hmac-private-key-at-least-64-bytes'
  serverSideSessionKeyHashPepper: 'your-session-pepper'
  emailSecretHashPepper: 'your-email-pepper'
  googleClientSecret: |
    {"web": {"client_id": "your-client-id", "client_secret": "your-client-secret",
    "redirect_uris": []}}

```

If your legacy project set `email_password_salt` or `emailPasswordPepper`, keep those keys, because the password import reads them to check legacy passwords.

See [Storing secrets](#) for production handling and additional provider-specific secrets. For the `googleClientSecret` format, see [Store your credentials](#).

The server now starts with both legacy and modular endpoints mounted. `enableLegacyClientSupport` forwards the legacy `email`, `status`, and `user` endpoints to the bridge. For old clients, this means:

- **Email sign-in** works through the bridge.
- **Account creation, password changes, and password resets** are stubs that return `false` or `null`.
- **Apple, Firebase, and Google sign-in** still reach the legacy module, like every other legacy endpoint that isn't forwarded.

Run the migration

Create and apply the schema migrations for the new modular tables:

```
serverpod create-migration --tag modular-auth
dart run bin/main.dart --role maintenance --apply-migrations
```

The `--role maintenance` flag makes the server exit once the migrations are applied.

Then run the user migration once. The example below migrates every legacy user, but you can pass `maxUsers` to process in batches if your dataset is large. The `userMigration` callback fires once per migrated user so you can remap your own foreign keys from the legacy `int` user ID to the new `UuidValue` auth user ID inside the same transaction.

Put this helper in its own file so its imports do not collide with `server.dart`.

`serverpod_auth_migration_server` exports its own `Endpoints` and `Protocol` classes, so use a `hide` clause when adding it elsewhere:

```
// lib/src/auth_migration.dart
import 'package:serverpod/serverpod.dart';
import 'package:serverpod_auth_idp_server/core.dart';
import 'package:serverpod_auth_migration_server/serverpod_auth_migration_server.dart'
  hide Endpoints, Protocol;

Future<void> runMigration(Serverpod pod) async {
  AuthMigrations.config = AuthMigrationConfig(
    emailIdp: AuthServices.instance.emailIdp,
  );

  final session = await pod.createSession();
  try {
    final migrated = await AuthMigrations.migrateUsers(
      session,
      userMigration: (
        session, {
```

```

        required int oldUserId,
        required UuidValue newAuthUserId,
        Transaction? transaction,
    }) async {
        // Remap your own tables from oldUserId to newAuthUserId using transaction.
    },
);
session.log('Migrated $migrated users.', level: LogLevel.info);
} finally {
    await session.close();
}
}

```

Run this as a one-off from a dedicated entry point, not on a request path. The simplest pattern is a script under `<project>_server/bin/migrate.dart` that imports the helper above and invokes it once on a started `Serverpod` instance. Don't start that instance with `--role maintenance`, because a maintenance process exits on its own and can cut the migration short. Do not call `runMigration` from inside an endpoint or future call.

The migration is idempotent: re-running `migrateUsers` skips users that already have a row in `serverpod_auth_migration_migrated_user`. The returned count is "users selected this run," not "new users created."

Wire up sign-in for migrated users

For email accounts, the `PasswordImportingEmailIdpEndpoint` from the server step runs the base login first. If the login fails with `invalidCredentials` and the account has no password yet, the endpoint imports the legacy password and retries. The bridge upgrades the password hash and removes the `LegacyEmailPassword` row. A rate-limited login fails with `tooManyAttempts` and never reaches the import.

Password import only works with the default legacy hashing. If your legacy `AuthConfig` set `extraSaltyHash: false`, a custom `passwordHashGenerator`, or a custom `passwordHashValidator`, the bridge can't import those passwords, and those users must reset their password.

⚠ WARNING

Google, Apple, and Firebase accounts aren't linked automatically:

- **Google:** the legacy Google sign-in stored the user's email as the identifier, and `migrateUsers` copies it unchanged. `AuthBackwardsCompatibility.importGoogleAccount` matches only the Google user ID, so it never finds those rows. On first sign-in, a legacy Google user gets a new, empty account. To keep their data, link the account by email in a [Google endpoint](#) that extends `GoogleIdpBaseEndpoint`.

- **Apple and Firebase:** the bridge has no import helper for these users, so they get no automatic linking.

A migrated email user can now sign in with their old password.

Update the Flutter app

Where your app creates the `Client` (`lib/client.dart` in projects created with 4.0, otherwise `lib/main.dart`), swap the auth setup to use `FlutterAuthSessionManager` and call `initAndImportLegacySessionIfNeeded` before any sign-in UI renders. This exchanges any old auth key stored on the device for a new modular session so existing installs do not have to sign in again.

```
import 'package:serverpod_auth_bridge_flutter/serverpod_auth_bridge_flutter.dart';
import 'package:serverpod_auth_core_flutter/serverpod_auth_core_flutter.dart';
import 'package:your_client/your_client.dart';

late Client client;

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();

  client = Client('https://api.example.com/')
    ..authSessionManager = FlutterAuthSessionManager();

  await client.authSessionManager.initAndImportLegacySessionIfNeeded(
    client.modules.serverpod_auth_bridge,
  );

  runApp(const MyApp());
}
```

If your app creates the client in `lib/client.dart`, add the `serverpod_auth_bridge_flutter` import there. Inside `initializeClient()`, replace `unawaited(client.auth.initialize());` with this call, which runs `initialize()` itself:

```
await client.authSessionManager.initAndImportLegacySessionIfNeeded(
  client.modules.serverpod_auth_bridge,
);
```

This requires `serverpod_auth_bridge_client` and `serverpod_auth_bridge_flutter` in `<project>_flutter/pubspec.yaml` from [Add the new auth packages](#).

Flutter clients that used the default legacy session storage carry the legacy session forward on first launch after the upgrade. If your project customized session storage, pass a `LegacyStringGetter` to `initAndImportLegacySessionIfNeeded` that reads from your custom location.

Verify and clean up

Verify

- Query `SELECT count(*) FROM serverpod_auth_migration_migrated_user` and confirm it matches your legacy user count.
- Sign in with a known migrated account through the new IdP and confirm the `LegacyEmailPassword` row for that user is gone afterwards.
- Over the following days, watch the `serverpod_auth_bridge_email_password` row count drop as email users sign in, and check whether requests still reach `serverpod_auth.*` endpoints. Don't track the other bridge tables' row counts, because they don't reliably shrink as users move.

While clients catch up

`LegacySessionTokenManager` validates legacy session tokens against the bridge's stored sessions. The legacy endpoints that `enableLegacyClientSupport` doesn't forward still answer only because the legacy module is still installed.

Once no client signs in through the legacy Apple, Firebase, or Google endpoints, refuse the unforwarded endpoints, because those sign-ins issue legacy session keys the new stack doesn't know:

```
pod.enableLegacyClientSupport(blockUnbridgedAuthEndpoints: true);
```

The legacy database tables stay untouched, so you can still roll back to the legacy stack. A rollback needs your own tables to resolve legacy `int` user IDs again. Either keep the legacy ID next to the new one in `userMigration`, or reverse the remap with the `serverpod_auth_migration_migrated_user` table. A rollback also loses everything created only in the new stack, such as new accounts, password changes, and linked Google accounts.

When you are ready to remove legacy

Remove the legacy packages once your clients have moved to the new stack. Work through these steps in order:

1. Remap your own fields and relations off the legacy integer user IDs, and drop the columns that still reference `userInfoId`.
2. Remove the code that uses bridge and migration symbols: `LegacySessionTokenManager`, `enableLegacyClientSupport`, the `runMigration` helper, `bin/migrate.dart`, and `initAndImportLegacySessionIfNeeded`. Replace the password-importing endpoint with a plain endpoint that extends `EmailIdpBaseEndpoint`.
3. Remove the legacy, bridge, and migration packages from the server `pubspec.yaml`, then from the client, then from the Flutter app.

4. Create a migration. It drops the legacy, bridge, and migration tables, so `serverpod create-migration` aborts with a warning until you pass `--force`. See [Force create migration](#).

Once that migration is applied, users still on a legacy session must sign in again. Email users who never signed in after the migration have no password, so they must reset it.

Troubleshooting

Symptom	Likely cause	What to do
<code>migrateUsers</code> throws or rolls back	Migrations not applied, or the wrong <code>emailIdp</code> instance on <code>AuthMigrationConfig</code>	Apply all module migrations; set <code>AuthMigrations.config = AuthMigrationConfig(emailIdp: AuthServices.instance.emailIdp)</code> after <code>pod.initializeAuthServices</code> .
Migrated email user cannot log in with their old password	<code>importLegacyPasswordIfNeeded</code> is not on the login path	Confirm your email endpoint subclasses <code>EmailIdpBaseEndpoint</code> and catches <code>EmailAccountLoginException</code> with reason <code>invalidCredentials</code> . For an account without a password, it must call <code>AuthBackwardsCompatibility.importLegacyPasswordIfNeeded</code> and then <code>super.login</code> again.
Flutter app prompts the user to sign in again after upgrade	<code>initAndImportLegacySessionIfNeeded</code> didn't run before the sign-in UI, <code>migrateUsers</code> hasn't run, or the app stores its session in a custom location	Call <code>initAndImportLegacySessionIfNeeded</code> before any sign-in UI renders. Run <code>migrateUsers</code> , so the bridge holds the old sessions. For custom session storage, pass a <code>LegacyStringGetter</code> that reads from it.
Duplicate Google <code>AuthUser</code> created on first sign-in	Legacy Google rows store the email, and <code>importGoogleAccount</code> matches only the Google user ID	Link legacy Google accounts by email before the base login. Re-running <code>migrateUsers</code> does not help, because it skips users who are already migrated.
Authentication handler rejects	<code>pod.initializeAuthServices</code> was not	Call <code>pod.initializeAuthServices(...)</code> before <code>pod.start()</code> ; include

Symptom	Likely cause	What to do
modular tokens	called, or token managers list is missing	<code>JwtConfigFromPasswords</code> (or <code>ServerSideSessionsConfigFromPasswords</code>) plus <code>LegacySessionTokenManager</code> in <code>tokenManagerBuilders</code> .
<code>serverpod generate</code> fails with "Endpoint analysis skipped due to invalid Dart syntax" or "The function 'Protocol' isn't defined"	The bridge or migration package exports its own <code>Endpoints</code> and <code>Protocol</code> classes that clash with your project's	Import the bridge with a <code>show</code> clause (<code>show LegacySessionTokenManager, LegacyClientSupport</code>) in <code>server.dart</code> , move the email endpoint subclass into its own file under <code>lib/src/endpoints/</code> , and use <code>hide Endpoints, Protocol</code> when importing the migration package in a helper file.
<code>PasswordNotFoundException: jwtRefreshTokenHashPepper was not found</code> on startup	<code>JwtConfigFromPasswords</code> requires <code>jwtRefreshTokenHashPepper</code> and <code>jwtHmacSha512PrivateKey</code> in <code>passwords.yaml</code>	Add both keys to each environment section. The same error for another key means another builder is missing its key, for example <code>googleClientSecret</code> for <code>GoogleIdpConfigFromPasswords</code> .

Still stuck?

Reach out on the [community page](#).

Related

- [Upgrade to 4.0](#): do this first.
- [Authentication setup](#): modular configuration reference.
- [Database migrations](#): creating and applying schema changes safely.